

Temporal Difference (TD) learning →

✗ let's first ~~do~~ do small modification to our Monte Carlo policy evaluation method!

$$V_{\pi}(s) \doteq E_{\pi}[G_t | S_t = s]$$

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)] \quad \equiv \text{incremental MC update rule!}$$

⇒ To compute the return we have to take samples of full trajectories. This means we don't learn during the episode. But we want to learn incrementally before the end of the episode.

Bootstrapping →

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

$$\begin{aligned} V_{\pi}(s) &\doteq E_{\pi}[G_t | S_t = s] \\ &= E_{\pi}[R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= R_{t+1} + \gamma V_{\pi}(s_{t+1}) \end{aligned}$$

Using this,

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)]$$

$$\cancel{V(s_t) \leftarrow V(s_t) + \alpha[R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]}$$

$$G_t \approx R_{t+1} + \gamma V(s_{t+1}) \}$$

$$V(s_t) \leftarrow V(s_t) + \alpha [R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Here we can think the value of the next state as a stand-in for the return until the end of the episode.

$\Rightarrow$ We call $\underbrace{R_{t+1} + \gamma V(s_{t+1})}$ as target!

$\Rightarrow R_{t+1} + \gamma V(s_{t+1}) - V(s_t) \equiv$ ~~TD error~~ TD error (denoted by δ_t)

$\Rightarrow$ The target of our update might seem a little strange at first. TD updates the value of one state towards its own estimate of value ~~of~~ next state in the

$\Rightarrow$ As the estimated value for the next state improves, so does our target. We have done something like this before in dynamic programming

$\Rightarrow$ In DP, we updated it towards the value of all possible next states.

$$TD \quad \left\{ V(s_t) \leftarrow V(s_t) + \alpha [R_{t+1} + \gamma V(s_{t+1}) - V(s_t)] \right.$$

$$DP \quad \left\{ V'(s) \leftarrow \sum_a \pi(a|s) \sum_{s', a'} p(s', a' | s, a) \cdot [\gamma + \gamma V(s')] \right.$$

⇒ The primary difference is in DP, we use an expectation over all possible next states.
 (we needed a model of environment to compute this expectation)

$$\sum_{s', r} p(s', r | s, a)$$

⇒ In TD, we only need the next state, we can get that directly from the environment without the model.

{ How we get that next state directly from the environment }? $\underline{\underline{S_t \leftarrow S_{t+1}}}$

Tabular TD(0) for estimating v_π :

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

Initialize S

Loop for each step of episode:

$A \leftarrow$ action given by π for S

Take action A , observe R, S'

$V(s) \leftarrow V(s) + \alpha [R + \gamma V(s') - V(s)]$

$S \leftarrow S'$

Until S is terminal.

⇒ TD learning: a way to incrementally estimate the return through bootstrapping.

Advantages of TD ⇒

- 1) TD elegantly combines key ideas from dynamic programming and monte carlo methods.
- 2) like DP, TD methods bootstrap.
- 3) like MC, TD can learn directly from experience.

⇒ We can learn online without waiting for the episode to end!

⇒ Unlike DP, TD methods do not require a model of the environment, they can learn directly from the experience.

⇒ Unlike MC, TD can update values at every step.
(Bootstrapping allows us to estimate based on other estimates)

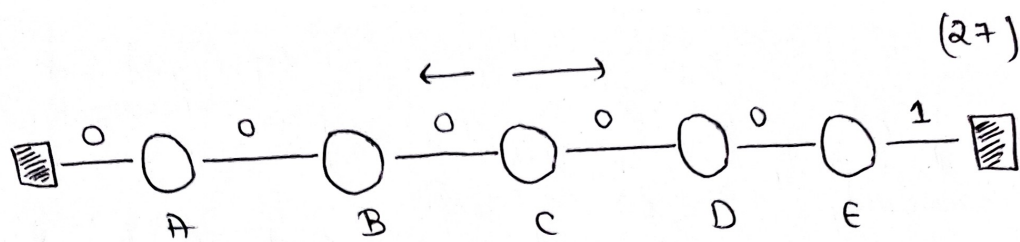
⇒ TD asymptotically converges to the correct predictions.

⇒ TD methods are usually faster than MC methods.

Example to Compare

TD and MC →

Random walk :-



let, $\pi(.|s) = 1/2, \forall s \in \mathcal{S}$

} uniform
random
policy

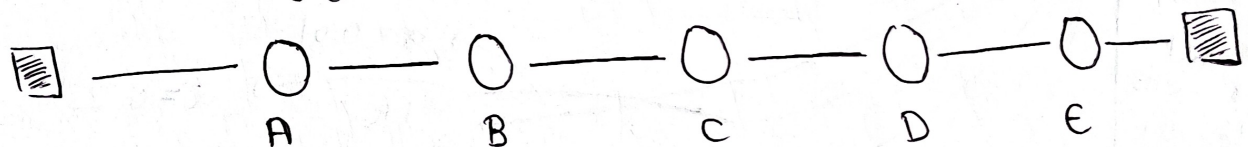
← we want to
evaluate
its value
function.

⊗ All episodes will start in C
and episodes will either terminate
on the left or on the right.

⊗ let $r=1$

⊗ In this problem, the value has an intuitive meaning. The
value of each state is the probability terminating
on the right when starting from each state.

Updates Using TD_v learning —
and MC



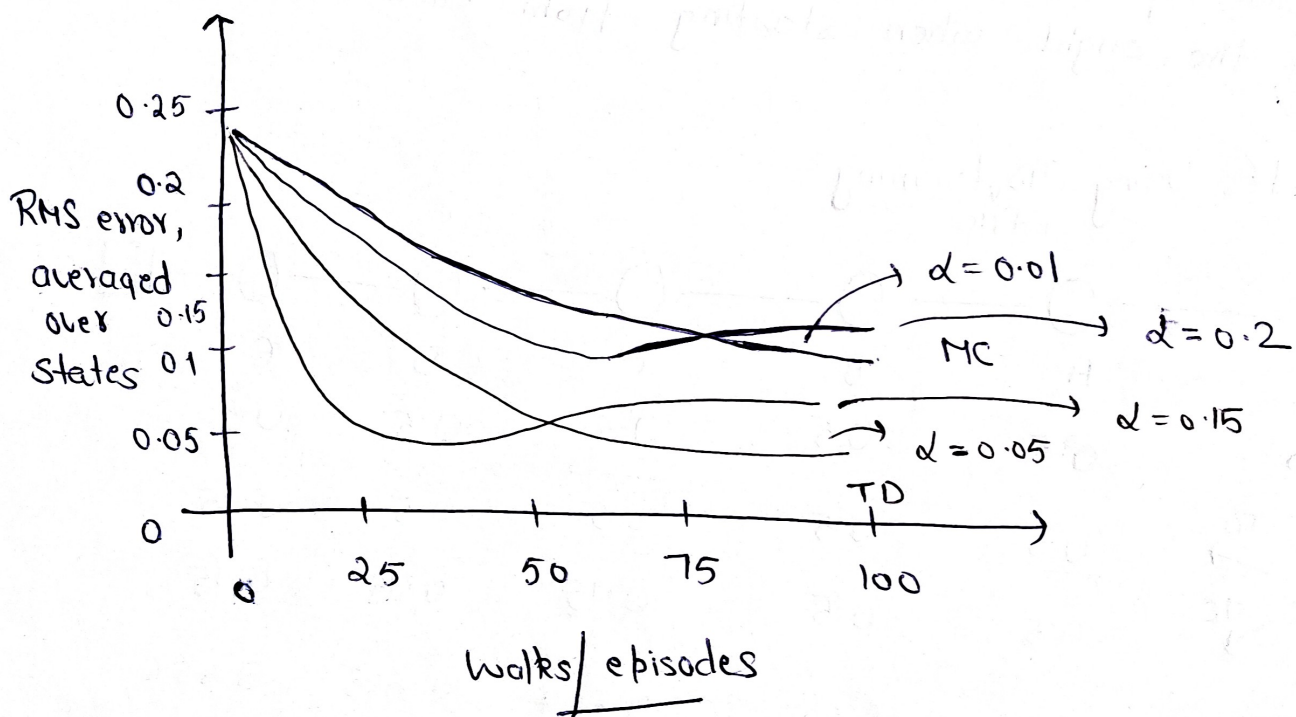
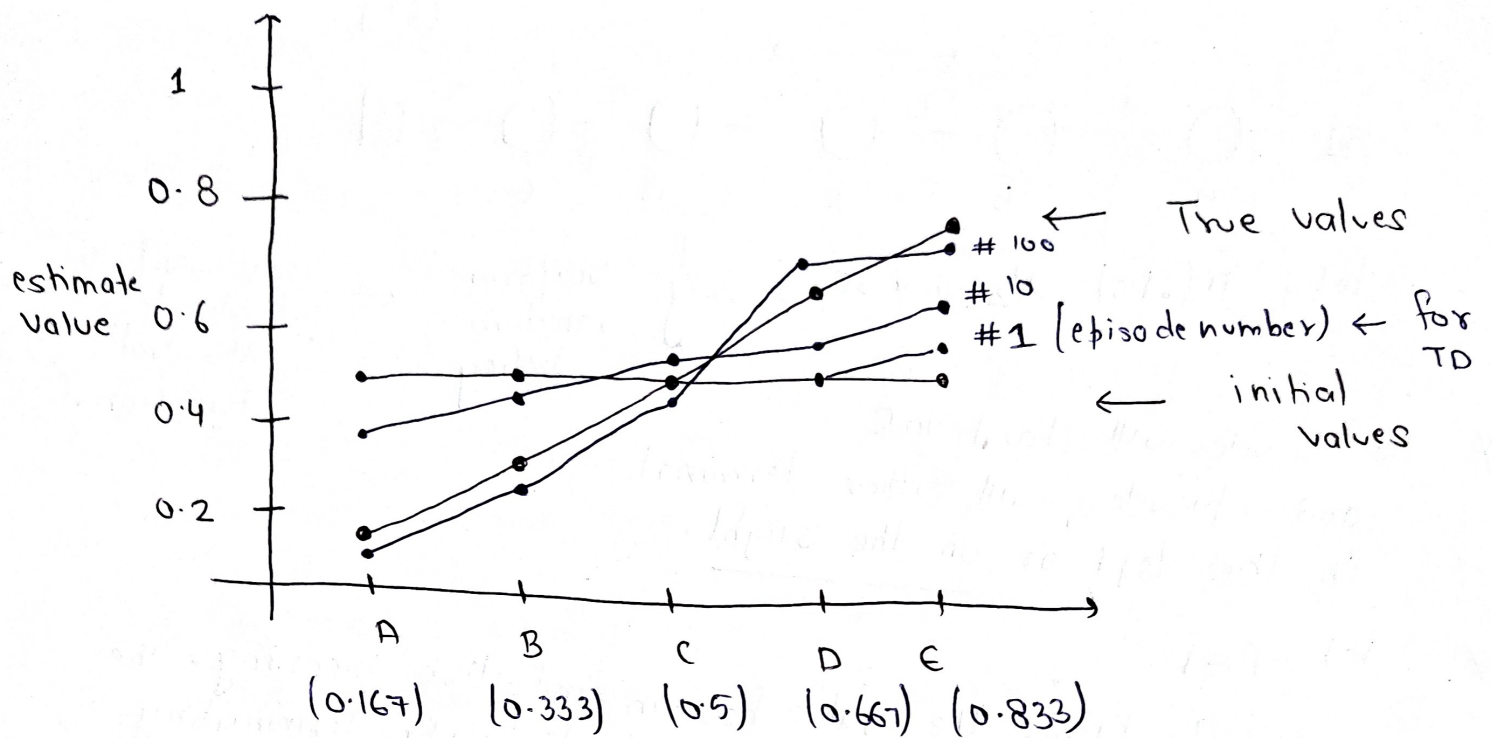
initialize →

end of episode 1
 $\xrightarrow{TD}$
 $\xrightarrow{MC}$

	A	B	C	D	E
initialize →	0.5	0.5	0.5	0.5	0.5
# end of episode 1 $\xrightarrow{TD}$	0.5	0.5	0.5	0.5	0.75
# end of episode 1 $\xrightarrow{MC}$	0.5	0.5	0.75	0.75	0.75

end of episode 2
 $\xrightarrow{TD}$
 $\xrightarrow{MC}$

	A	B	C	D	E
# end of episode 2 $\xrightarrow{TD}$	0.251	0.503	0.527	0.563	0.656
# end of episode 2 $\xrightarrow{MC}$	0.25	0.25	0.375	0.375	0.375



TD Converges faster to a lower final error!